



puzo903 / sever-side-web-application ▾



Code

Issues

Pull requests

Actions

Projects

Wiki

More ▾



master ▾

sever-side-web-application / README.md



puzo903 test(rpc): add model-based testing with hypothesis verifying RPC states

1d58c80 · 2 minutes ago ⌚

126 lines (102 loc) · 7.72 KB



Preview

Code

Blame

Raw



Практическая работа №1 — Вариант 18 (Этап 1)

1. Общее описание

В данном проекте реализована in-memory модель слоя работы с данными для прототипа веб-приложения. Сохранение данных на диск не производится. Для представления записей таблиц `Member`, `Instruction` и `Response` в памяти используются словари.

2. Описание всех функций и настроек

Модель предоставляет 13 функций для работы с сущностями:

- **Создание новой записи** (для `Member`, `Instruction`, `Response`).
- **Получение всех записей** (для всех сущностей).
- **Получение одной записи по идентификатору** (для всех сущностей).
- **Редактирование записи** (для всех сущностей).
- **Операция соединения (join)**: Выборка данных таблиц `Instruction` и `Response`, отфильтрованных по условию `created >= now - 9 min`.

Для взаимодействия с моделью реализован интерактивный консольный интерфейс (REPL). Сторонние настройки среды не требуются.

3. Описание команд для сборки проекта и запуска тестов

Для запуска скрипта в интерактивном режиме (REPL) выполните следующую команду в корне репозитория: `python src/app.py`

4. Примеры использования

Пример сессии работы в режиме REPL, демонстрирующий создание, просмотр, редактирование и объединение записей:

```
> create member 192.168.1.1 Mozilla
> create instruction arg_1 0 desc_1 active
> create response ok ok no_error 0 1 100
> get_all members
> edit member 0 10.0.0.1 Safari
> get member 0
> join
> exit
```



Этап 2. Удалённый вызов процедур на основе TCP

Практическая работа №1 — Вариант 18 (Этапы 1 и 2)

1. Общее описание

Реализован прототип веб-приложения с поддержкой удалённого вызова процедур (RPC) на основе протокола TCP. В памяти (через словари) хранятся данные сущностей `Member`, `Instruction` и `Response`.

2. Описание всех функций и настроек

Сервер поддерживает 13 функций для CRUD-операций и выборки (`join`). **Настройки RPC:**

- Формат сериализации тела сообщений: **XML**.
- Порядок байт: **от младшего к старшему (little-endian)**.
- Структура запроса: Размер тела (4 байта) + Код операции (1 байт) + XML.

- **Структура ответа:** Код операции (2 байта) + Размер тела (3 байта) + XML.
- **Журналирование:** Все ответы от RPC записываются в файл `journal.log`.

3. Описание команд для сборки проекта и запуска тестов

Запуск TCP RPC сервера: `python src/server.py`

Запуск клиента (в отдельном окне терминала): `python src/client.py`

4. Примеры использования

Пример программного вызова удаленной процедуры через клиентский класс:

```
from client import RPCClient

client = RPCClient('127.0.0.1', 8000)
member = client.create_member("127.0.0.1", "Chrome")
print(client.get_all_members())
```



Практическая работа №1 — Вариант 18 (Итоговый)

1. Общее описание

Прототип веб-приложения с поддержкой удалённого вызова процедур (RPC) на основе протокола TCP. Реализовано in-memory хранение данных (сущности `Member`, `Instruction` и `Response`) в виде словарей без сохранения на диск. Обеспечено стопроцентное покрытие целевого кода системы автоматическими тестами с использованием подхода Model-Based Testing (MBT).

2. Описание всех функций и настроек

Сервер предоставляет 13 функций для работы с сущностями:

- **CRUD-операции:** Создание, чтение всех, чтение одной по ID и редактирование записей для каждой из трех сущностей.
- **Операция соединения (join):** Выборка данных `Instruction` и `Response`, отфильтрованных по условию `created >= now - 9 min`.

Настройки сети и тестирования:

- **Протокол:** TCP с кастомной обработкой фрагментации пакетов (`recvall`).
- **Сериализация:** Формат тела сообщений — **XML**.
- **Порядок байт:** От младшего к старшему (**little-endian**).
- **Структура запроса:** Размер тела (4 байта) + Код операции (1 байт) + XML-тело.
- **Структура ответа:** Код операции (2 байта) + Размер тела (3 байта) + XML-тело.
- **Журналирование:** Код операции и размер тела всех ответов RPC фиксируются в `journal.log` .
- **Тестирование (MBT):** Автоматическая генерация сценариев с помощью `RuleBasedStateMachine` (библиотека `hypothesis`). Замер покрытия ветвей кода с помощью утилиты `coverage` .

3. Описание команд для сборки проекта и запуска тестов

Для работы с локальной in-memory моделью через интерактивный консольный интерфейс (REPL):

```
python src/app.py
```



Для ручного запуска TCP RPC сервера:

```
python src/server.py
```



Для автоматического запуска MBT-тестов и оценки покрытия ветвей используйте подготовленный bash-скрипт:

```
bash run.sh
```



4. Примеры использования

Пример работы в режиме REPL:

```
> create member 192.168.1.1 Mozilla
> create instruction arg_1 0 desc_1 active
> create response ok ok no_error 0 1 100
> join
> get_all members
> exit
```



Вывод консоли после успешного запуска тестов (со 100% покрытием бизнес-логики):

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src/app.py	65	0	18	0	100%	
src/client.py	35	0	0	0	100%	
src/server.py	40	0	4	0	100%	
tests/test_model.py	76	0	18	0	100%	
TOTAL	216	0	40	0	100%	

